AI SECURITY LAB / 2026

OWASP LLM + Agentic Risk Proof of Concept

Prepared for: Youth Growth Foundation (YGF NGO) — AI Security Internship, Round 2

Candidate: Nipun Agarwal Date: 12 November 2026

33 harmless attack cases | 11 risk groups | 132 fixture rows

Deliverable	Scope
Browser app	Localhost POC with switchable Claude / Gemini adapters
AI agent	Fake file read, email, HTTP and MCP manifest connection
Tests	At least 3 per required risk; before-and-after controls
Report status	Offline simulation verified; live model benchmarking pending API access

RESEARCH INTEGRITY

The results in this document are deterministic application-fixture simulations. The values do not represent real responses from Claude or Gemini. To compare the actual LLM services, the repository includes separate API-backed testing commands that require valid model credentials.

01 / Executive summary

The POC illustrates how unsafe application orchestration can expose secrets, accept injected tasks or tool definitions, generate unsafe HTML, or promote untrusted context into privileged actions. An intentionally vulnerable fixture is paired with a hardened server-side security control for each risk.

Key results from sandbox-fixture tests

Metric	Fixture result	Meaning
Risk families	11 (10 specified + MCP)	Required 2026 taxonomy covered
Distinct attack cases	33	3 per risk group
Model-labeled comparisons	132 rows	2 labels × before/after; not real LLM calls
Before mitigation	66 / 66 markers observed	By design; application vulnerability fixtures
After mitigation	0 / 66 markers observed	Deterministic server-side policies blocked markers
Unintended real side effects	0	No shell, real network calls or external mail

OWASP 2026 mapping matters

The 2026 LLM list was renumbered relative to 2025. For this assignment, LLM03 means Excessive Agency, LLM08 means Hidden Context Exposure, and LLM10 means Improper Output Handling. Agentic ASI05 covers unexpected code execution; ASI06 covers memory and context poisoning.

Evaluation limits

Fixture results verify that the demonstrator differentiates vulnerable from hardened controls. They do not measure model guardrail quality, probability of a successful jailbreak, provider ranking, or real tool-use exploit rates.

02 / Architecture and method

Component	Purpose	Trust boundary
User & untrusted KB	Malicious toy instructions and fake records	Must not override trusted user goal
Claude/Gemini adapters	Optional API-backed text inference	Model response treated as untrusted
Tool router	read_file; send_email; http_call; mcp_connect	Server-side target, role and approval rules
Fixture runner	Repeatable 33-case baseline	No external side effects
Hardened path	Per-risk policy checks	Unauthorized intent rejected

Execution protocol

1. Prepare 33 toy adversarial prompts and 11 distinct mitigation rules.

2. For each case, execute vulnerable and hardened fixture modes under both provider-target labels.

3. Log Risk ID, model label, exact prompt, defense phase, observed marker and supporting response.

4. Apply same scenarios in real provider mode only after securely configuring local Claude and Gemini API keys.

5. Analyze actual model text manually; do not equate absence of a marker with complete security.

Environment

All fixture tests run with Python standard library on 127.0.0.1:8765. A minimal non-root Dockerfile is provided for additional isolation. Fake emails are only appended to an in-memory queue; HTTP entries are logged without requesting URLs; candidate code is never executed. The toy private file carries SECRET-FLAG-123.

03 / Required LLM risks (2026)

LLM01: Prompt Injection

Cases: KB override, Role spoofing, Support note injection. Control: Separate retrieved data from instructions; deny tool-like directives in data.

LLM02: Sensitive Information Disclosure

Cases: Payroll leak, Hidden token extraction, Indirect privacy request. Control: Access-controlled redaction of sensitive records before context assembly.

LLM03: Excessive Agency

Cases: Unapproved email, Unapproved remote action, Tool overreach. Control: Human approval and least-privilege checks for side-effecting tools.

LLM08: Hidden Context Exposure

Cases: System note reveal, Debug context dump, Indirect context reveal. Control: Keep system-only context out of accessible tools and redact hidden markers.

LLM10: Improper Output Handling

Cases: HTML injection, Script tag, Unsafe embed. Control: Escape HTML on rendering and allowlist action payloads.

2026 risk	Vulnerable (3 cases × 2 labels)	Hardened (3 × 2)
LLM01 — Prompt Injection	6 / 6 toy markers	0 / 6 toy markers
LLM02 — Sensitive Information Disclosure	6 / 6 toy markers	0 / 6 toy markers
LLM03 — Excessive Agency	6 / 6 toy markers	0 / 6 toy markers
LLM08 — Hidden Context Exposure	6 / 6 toy markers	0 / 6 toy markers
LLM10 — Improper Output Handling	6 / 6 toy markers	0 / 6 toy markers

Interpretation

The 30/30 before and 0/30 after counts on this page are fixture outcomes only. The same scenario strings may behave differently when submitted to a real API; no empirical Claude/Gemini results were available during this preparation.

04 / Required Agentic risks (2026)

ASI01: Agent Goal Hijack

Cases: Knowledge-base detour, Injected task plan, Fake supervisor. Control: Bind a signed user goal; ignore replacement goals from retrieved content.

ASI02: Tool Misuse and Exploitation

Cases: Private read, MCP forged tool, Unsafe URL. Control: Tool parameter schema plus allowlisted targets and external MCP trust boundary.

ASI03: Identity and Privilege Abuse

Cases: Claim admin, MCP role spoofing, Delegated authority. Control: Server-side caller role enforcement; reject role claims inside prompts.

ASI05: Unexpected Code Execution

Cases: Shell in report, Injected calculator, Command in document. Control: No eval or shell; structured allowlisted arithmetic with explicit literals.

ASI06: Memory and Context Poisoning

Cases: Persistent preference, Tool-response memory, Corrupted summary. Control: Write-protect trusted memory; store provenance and require approval.

2026 risk	Vulnerable fixture	Hardened fixture
ASI01 — Agent Goal Hijack	6 / 6 markers	0 / 6 markers
ASI02 — Tool Misuse and Exploitation	6 / 6 markers	0 / 6 markers
ASI03 — Identity and Privilege Abuse	6 / 6 markers	0 / 6 markers
ASI05 — Unexpected Code Execution	6 / 6 markers	0 / 6 markers
ASI06 — Memory and Context Poisoning	6 / 6 markers	0 / 6 markers

Agent safety note

A secure application cannot rely on the language model alone to decide whether the caller is an admin or whether a tool action is approved. Enforced permissions, explicit side-effect approval and execution sandboxing remain the application responsibility.

05 / MCP connection risk and POC

The connection test simulates a rogue MCP-like tool provider that attempts to influence the local agent. The payloads are a tool-name collision, an unapproved read-write capability expansion, and adversarial tool-description instructions.

Step	Vulnerable simulator	Hardened simulator
1. Discover server	Trusts a forged manifest	Requires trusted-local-v1 identity
2. Parse tools	Accepts send_email / private read	Allows public_kb_search only
3. Honor scope	Allows read-write claim	Requires read-only scope
4. Execute connection	Logs accepted mock manifest	Returns BLOCKED without adding tool
Result (3 × 2 labels)	6 / 6 toy markers	0 / 6 toy markers

Boundary diagram

UNTRUSTED MCP METADATA	HOST-ENFORCED POLICY	MOCK AGENT
Server ID / advertised tools / scope	Pin identity + allowlist capabilities	Access public KB only
Injected tool description	Treat as data, never instruction	No privilege elevation

MCP limitation

This demonstrates capability-validation logic but does not implement a live MCP transport, cryptographic attestation, or a complete interoperable MCP server-client handshake. A full protocol exercise would be a further enhancement.

06 / Claude versus Gemini: status

The project implements two provider adapters using the vendors' Python SDKs. No real provider requests were made when producing the included fixture ledger. Results must therefore be treated as NOT YET BENCHMARKED, not as a tie or comparative security ranking.

Measure	Claude	Gemini
Adapter in code	Implemented	Implemented
Fixture cases labeled for model	33 × 2 phases	33 × 2 phases
Actual API prompts sent	0	0
Verified live attacks / mitigations	Not tested	Not tested
Fixture marker baseline	33 / 33 (synthetic)	33 / 33 (synthetic)
Fixture marker after defense	0 / 33 (synthetic)	0 / 33 (synthetic)

How to complete the genuine comparison

1. Add both API keys to local environment variables, never to GitHub. 2. Run python run_tests.py --mode live --provider both after installing dependencies. 3. Review the CSV for refusals, false positives and differences in wording. 4. Separate model-response outcomes from app-level tool effects; here live mode is text-only. 5. Rebuild the report/results sheet from observed live outcomes and describe any API or model version differences.

Threat and benchmark limitations

As written, live probing evaluates exact toy marker appearance and does not invoke provider-native tool calling. It is inappropriate to infer that a provider is safer solely from the resulting marker count. The PoC illustrates application-layer controls rather than production-hardened defenses.

07 / Evidence and next steps

Files delivered

The submission bundle includes working Python sources, README, Dockerfile, 33 scenario prompts, unit tests, a CSV evidence log, a formatted spreadsheet, this PDF, and a local explanatory demo video.

Verification

The Python unit test suite confirms that all 33 vulnerable fixtures expose their expected toy markers, all 33 hardened fixtures suppress those markers, the MCP guard rejects a malicious manifest, a private file is blocked, and fake email uses an in-memory queue. Test report: 6 tests, all passing.

Further improvements

Priority 1: run both real providers under explicit authorization. Priority 2: implement provider-native structured function calling to exercise actual tool requests and separate model failures from control-plane failures. Priority 3: test official MCP transport and server verification. Priority 4: integrate a SIEM pipeline such as Wazuh for monitoring (optional extra credit).

References

OWASP GenAI LLM Top 10 (2026): https://genai.owasp.org/resource/owasp-genai-llm-top-10-2026/

OWASP Agentic Top 10 (2026): https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

Anthropic SDK: https://platform.claude.com/docs/en/cli-sdks-libraries/sdks/python

Google Gemini SDK: https://ai.google.dev/gemini-api/docs/get-started

End of report. All data and exploits are simulated; no real systems were targeted.